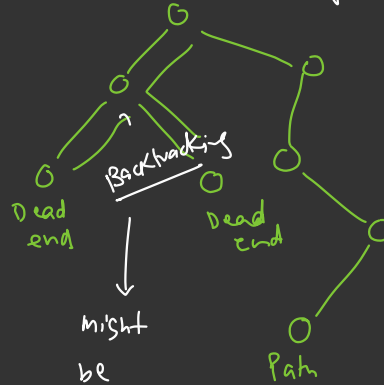
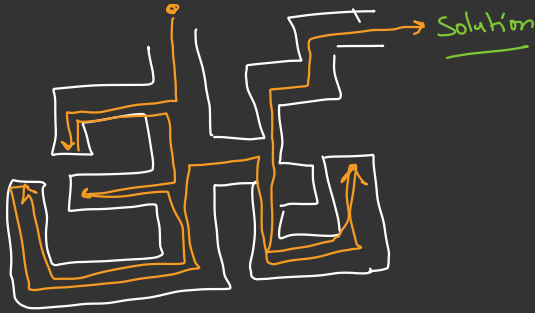


Backtracking

[Recursion & Backtracking]

Complete Search

- ↳ Explore entire search space
- ↳ generate one possible soln



{ some additional work while backtracking }

Example

1		
	3	
2		

Sudoku

1	2	3
2	3	1
3	1	2

1, 2, 3

abc

↳ abc
bac
cba
:

{ Backtracking }

↳ Print one soln (Sudoku)

↳ Printing "many solns" → Permutation

N "abc"

$(N!)$

Q generate all strings of len (N) with digits 4,5

$N=3$

$\begin{array}{ccc} _ & _ & _ \\ \downarrow & \downarrow & \downarrow \\ 4,5 & 4,5 & 4,5 \end{array}$

$\begin{array}{ccc} 4 & 4 & 4 \\ _ & _ & _ \\ 4 & 4 & 5 \\ _ & _ & _ \\ 4 & 5 & 5 \\ _ & _ & _ \\ 5 & 4 & 5 \\ _ & _ & _ \\ & & \end{array}$

$2 \times 2 \times 2 = 8$ combinations

char
generateStrings (n, i, output[]) {

// Base Case

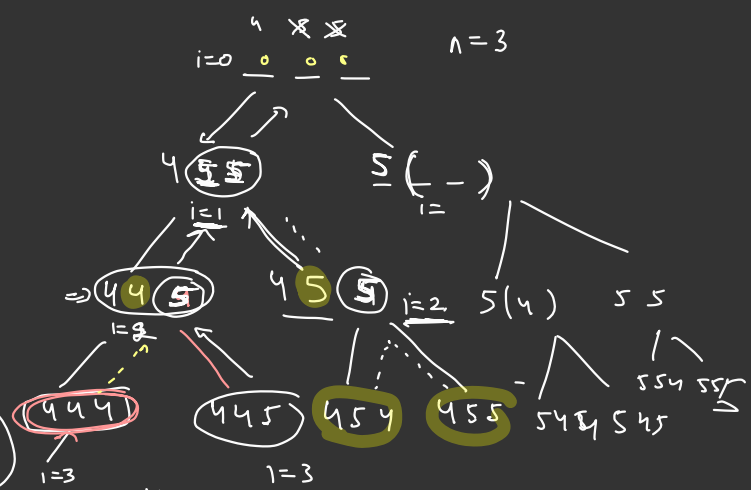
→ if (i == n)
 print (output)
 return

// Rec Case

l1 ⇒ output[i] = 4
l2 ⇒ generateStrings (n, i+1, output)
 output[i] = 0 ← (optional)
l3 ⇒ output[i] = 5
l4 ⇒ generateStrings (n, i+1, output)
 }

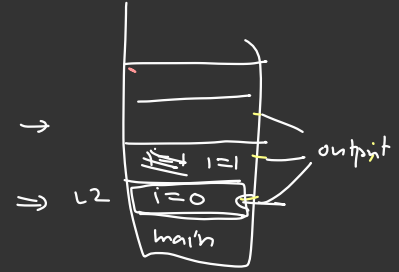
Backtracking

generateStrings (3, 0, output = [])



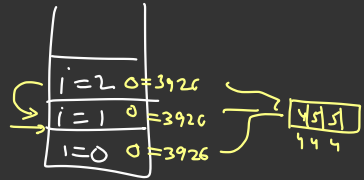
will get overwritten

444
445
454
455



Output → memory is shared across all fns

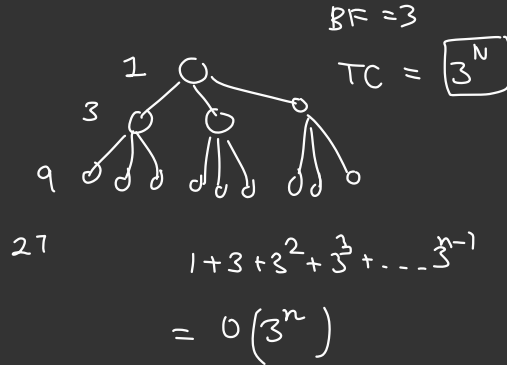
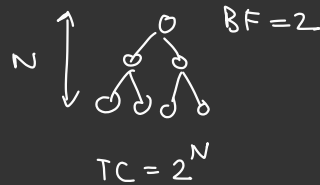
i → Primitive (Stack)



```

public static void generateStrings(int n,int i,char[] output){
    if(i==n){
        System.out.println(Arrays.toString(output));
        return;
    }
    //rec case
    // System.out.println(Arrays.toString(output));
    output[i] = '4';
    generateStrings(n,i+1,output);
    output[i] = '5';
    generateStrings(n,i+1,output);
    output[i] = '6';
    generateStrings(n,i+1,output);
}

```



Q generate all subsequences of a given string

$$\frac{2^{abc}}{2} = 8$$

a b c d e f

Q $\left[\begin{array}{c} b \\ c \\ bc \\ "" \end{array} \right] \left\{ \begin{array}{l} \leftarrow 1^{len} \\ \leftarrow 2^{len} \\ \leftarrow 0^{len} \end{array} \right\}$
4 subsets

+

~~Q~~ $\left[\begin{array}{c} b \\ c \\ bc \\ "" \end{array} \right] \left\{ \begin{array}{l} \leftarrow 1^{len} \\ \leftarrow 2^{len} \\ \leftarrow 0^{len} \end{array} \right\}$
4 subsets

output $\left[\begin{array}{c} ab \\ ac \\ abc \\ a \end{array} \right] \leftarrow$

$\left[\begin{array}{c} b \\ c \\ bc \\ "" \end{array} \right] \leftarrow$

generateSubsets (string input, int i, string output) → Stores one output

// Base Case

```
if (i == input.size()) {
    print(output)
    return;
}
```

// Rec Case

3

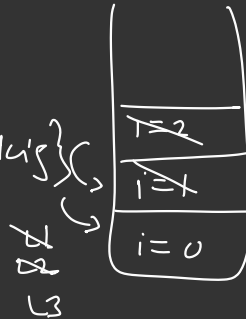
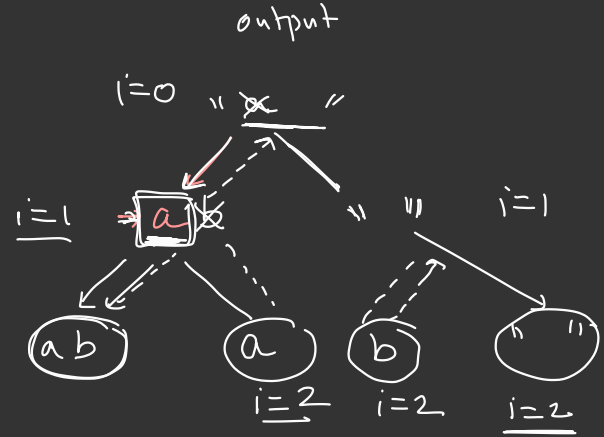
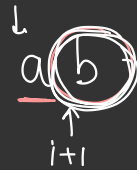
L1 → output.add(input[i]) — down

L2 → generateSubsets(input, i+1, output) — 2

L3 → output.removeLast() ← up {Backtracking}

⇒ L4 → generateSubsets(input, i+1, output)

}



→ ab ✓
→ a ✓
→ b ✓
→ "" ✓

outputs.

generateSubsets("ab", 0, "")

fn (string S)

⋮

f (S + "A")

f (S)

S is not modified

$$\left\{ \begin{array}{l} a = 5 \\ \Rightarrow a = a + 1 \leftarrow \\ f(a) \\ \Rightarrow a = a - 1 \\ f(a) \end{array} \right.$$

$$\left\{ \begin{array}{l} a = 5 \\ f(\overleftarrow{a+1}) \leftarrow \text{inc} \\ f(\underline{a}) \leftarrow \text{exc} \end{array} \right.$$

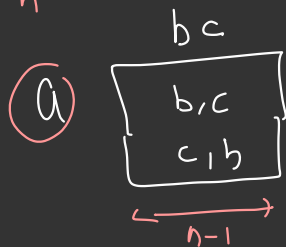
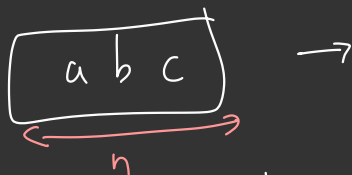
expression -

Break

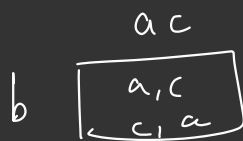


10.40

[Permutations]

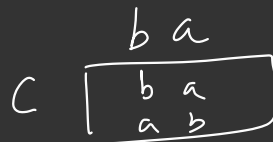


abc
acb

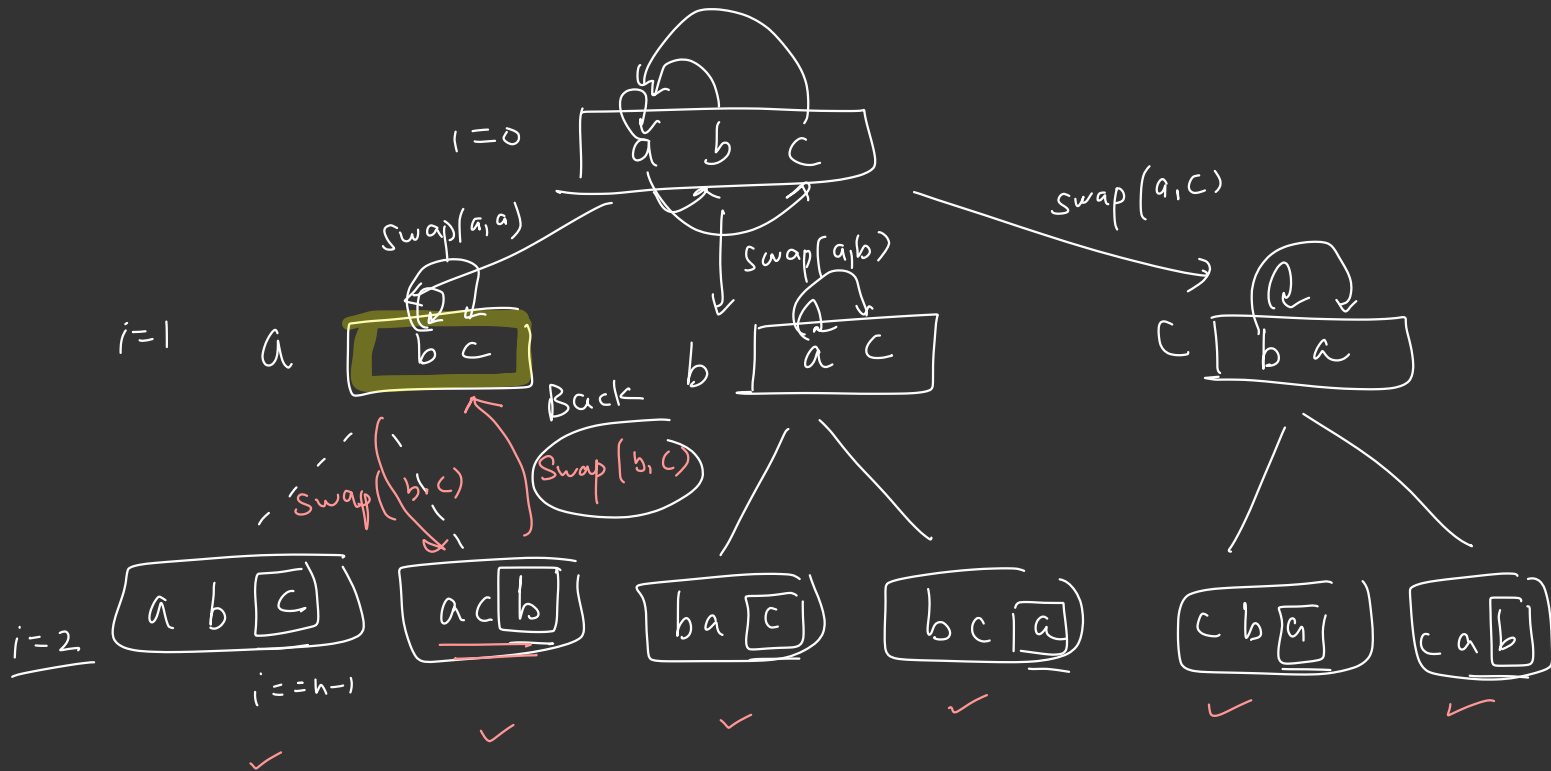


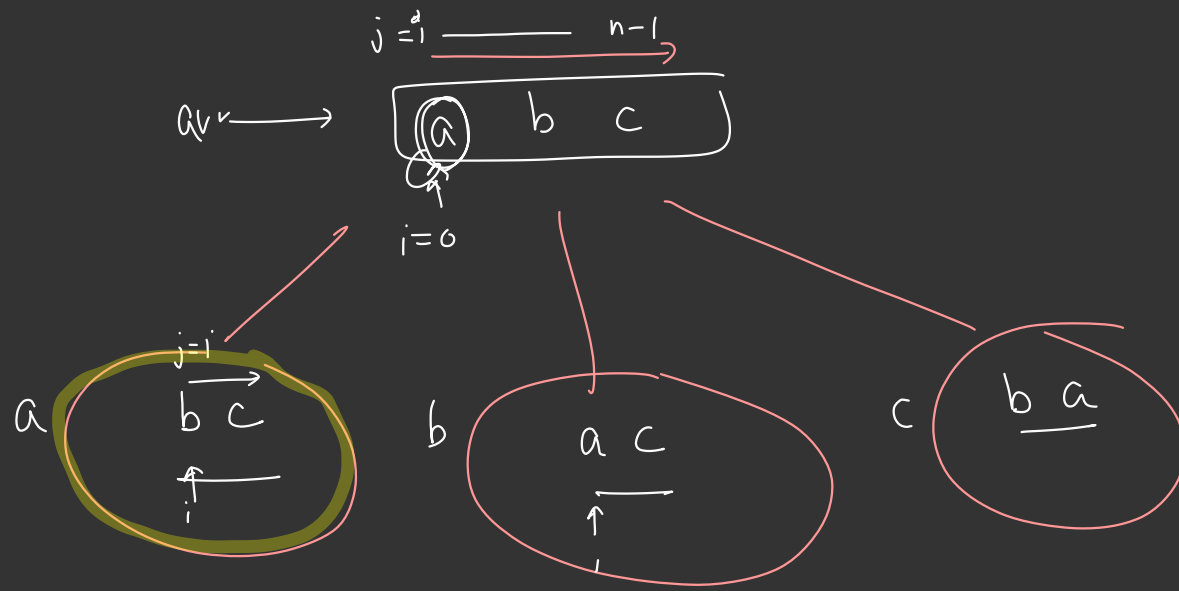
bac
bca

$3!$
 $=$



cba
cab





● `permute ( char[] input, i, n ) { i=0, j=1`

`if ( i == n-1 )`
`print ( input )`
`return,`

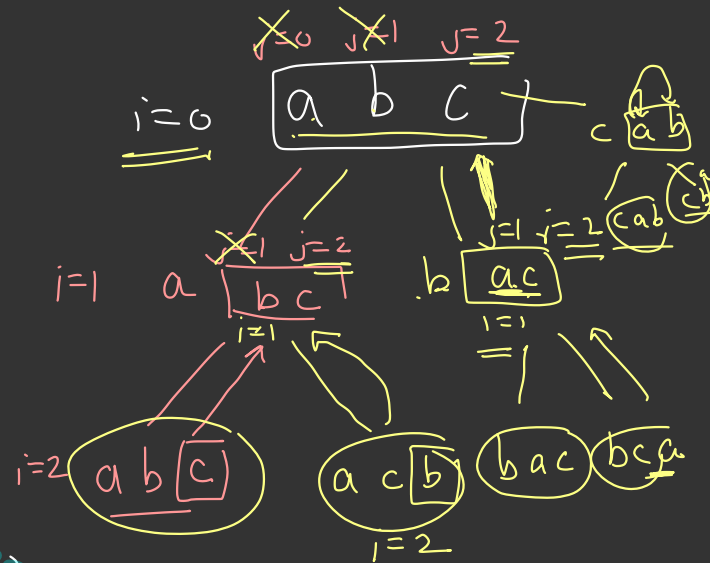
// Rec Case

`for ( j=i ; j <= n-1; j++ ) {`

`swap ( arr[i], arr[j] )`

`⇒ permute ( input, i+1, n )`

`3` `⇒ swap ( arr[i], arr[j] ) ← Restore the array while going Back`
`[ Backtracking ]`
 output }
`abc, acb`



Depth:

Space

$O(N)$

call
stack



abc

abc

acb

cab

cba

abc

ac**b**

(n!)

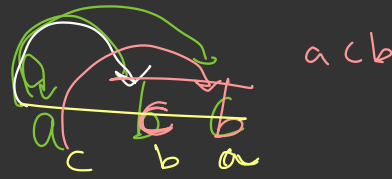
without Backtracking

Problem:

(abc, acb)

→ Repetition

→ Missing (bac, bca)



acb

a

cb

c

b

a

ab

cb

3 calls

$3 \times 2 \neq 6$
calls

j=0, ① ②

a b c

i=0

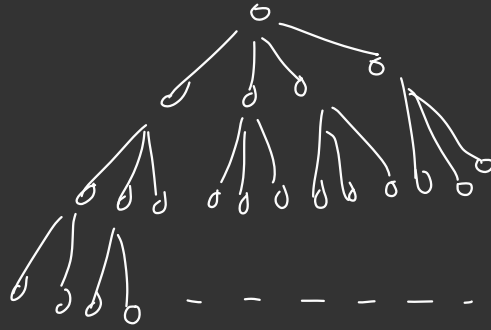
$i=i+1$

a b c
↑
i

b a c

c b a

a b c d



$$\underline{4 \times 3 \times 2 \equiv 1}$$

$$h = 4$$

$$n!$$

Permutations $\Rightarrow O(n!)$

X

X

Sunday (20 Nov) $\Rightarrow$ Extra class on

→ 9 00 PM

Backtracking

(optional)